Name : Vatsal Postiwala PRN No: 26070122509 Batch : B3(2025-2029)

ASSIGNMENT No: 27

TITLE: Develop a Java application demonstrating socket programming for communication between a client and a server.

Problem Statement

Develop a Java application demonstrating socket programming for communication between a client and a server.

Learning Objectives

To establish communication between client and server using Java sockets.

Theory

Socket programming enables communication between computers over a network using TCP/IP protocols. Java provides the Socket class for client-side communication and the ServerSocket class for server-side communication. A server listens for client requests on a specific port, while clients establish connections using the server's IP address and port number. Once connected, data is exchanged using input and output streams. Socket programming supports reliable communication between distributed systems and is widely used in chat applications, online gaming, file transfer systems, remote monitoring, and client-server architectures.

Code:

Server.java

```
package E27;

import java.lang.*;
import java.io.*;
import java.net.*;

public class Server {
  public static void main(String[] args) throws Exception {
    ServerSocket ss = new ServerSocket(4000);
    System.out.println("Server is waiting for connection...");
    Socket s = ss.accept();
    System.out.println("Client connected successfully!");

    s.close();
    ss.close();
  }
```

}

Client.java

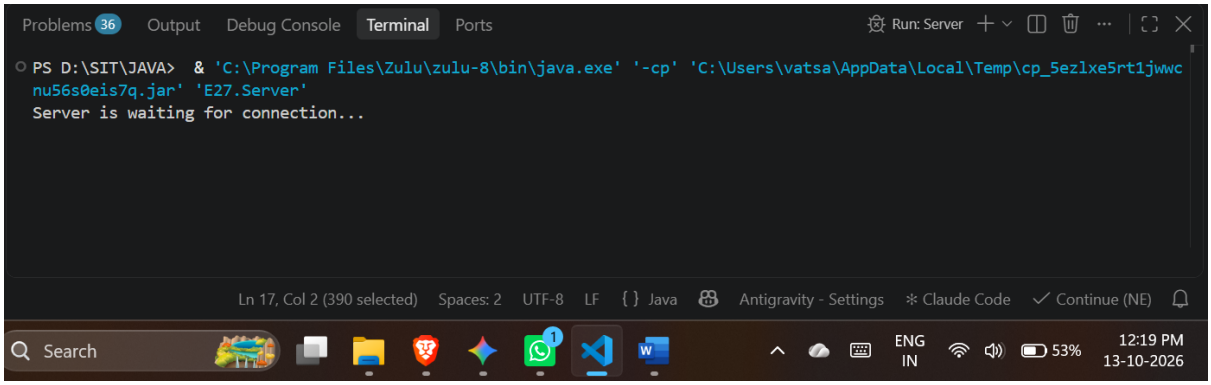
```
package E27;

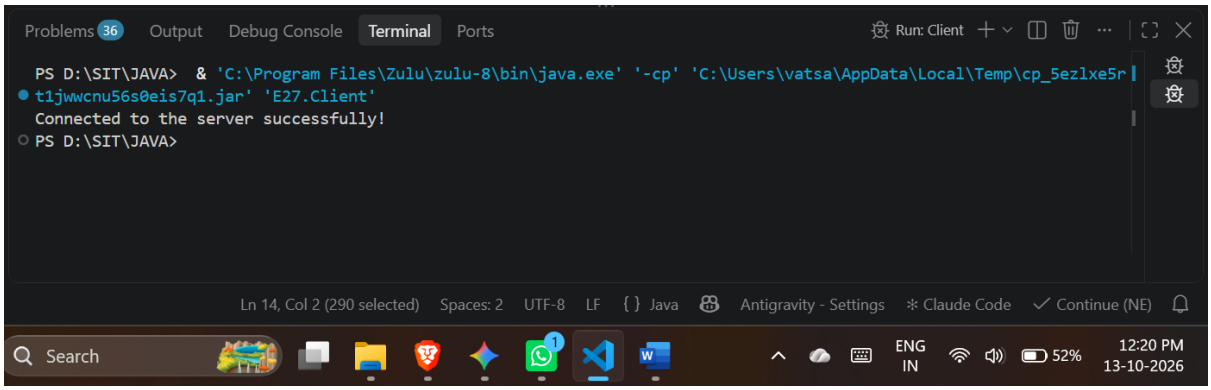
import java.lang.*;
import java.io.*;
import java.net.*;

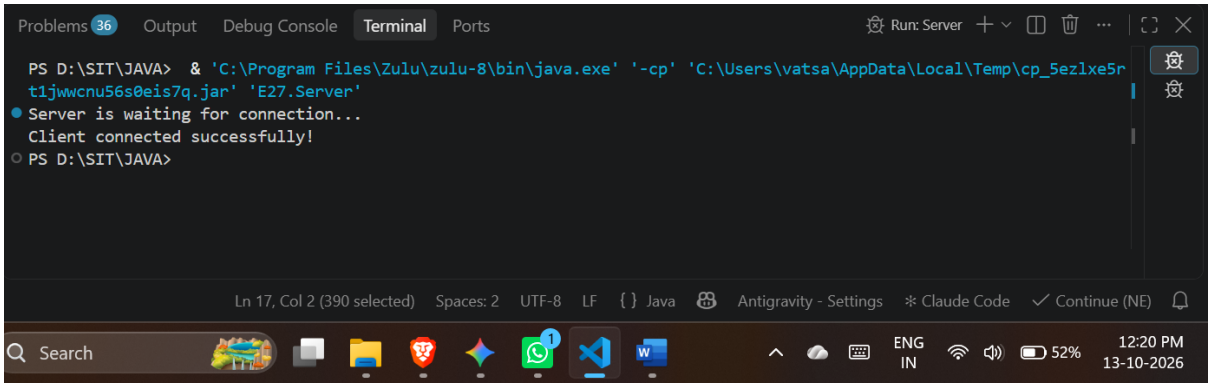
public class Client {
  public static void main(String[] args) throws Exception {
    Socket s = new Socket("localhost", 4000);
    System.out.println("Connected to the server successfully!");

    s.close();
  }
}
```

Output:







Exercise

1. Create a simple **client-server chat application** using sockets.

Code:

SimpleChatServer.java

```
package E27;

import java.lang.*;
import java.io.*;
import java.net.*;

public class SimpleChatServer {
   public static void main(String[] args) throws Exception {
      ServerSocket ss = new ServerSocket(6000);
      System.out.println("Server waiting for connection...");
      Socket s = ss.accept();
      System.out.println("Client connected! Type 'exit' to stop.");

                        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
      PrintWriter out = new PrintWriter(s.getOutputStream(), true);
                        BufferedReader kb = new BufferedReader(new
InputStreamReader(System.in));

      while (true) {
         String clientMsg = in.readLine();
         if (clientMsg == null || clientMsg.equalsIgnoreCase("exit")) {
            break;
         }
         System.out.println("Client: " + clientMsg);

         System.out.print("Server (You): ");
```

```
            String serverMsg = kb.readLine();
            out.println(serverMsg);
            if (serverMsg.equalsIgnoreCase("exit")) {
                break;
            }
        }

        s.close();
        ss.close();
    }
}
```

SimpleChatClient.java

```
package E27;

import java.lang.*;
import java.io.*;
import java.net.*;

public class SimpleChatClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 6000);
        System.out.println("Connected to server! Type 'exit' to stop.");

                        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
                        BufferedReader kb = new BufferedReader(new
InputStreamReader(System.in));

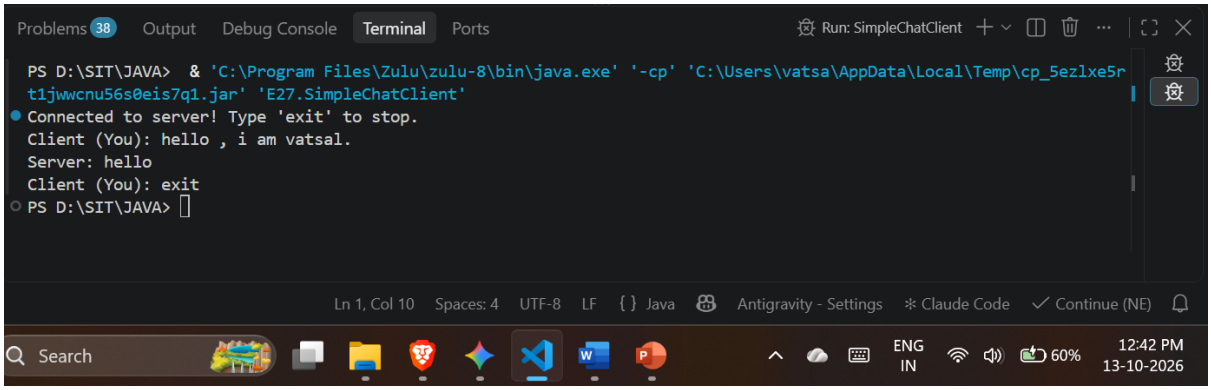
        while (true) {
            System.out.print("Client (You): ");
            String clientMsg = kb.readLine();
            out.println(clientMsg);
            if (clientMsg.equalsIgnoreCase("exit")) {
                break;
            }

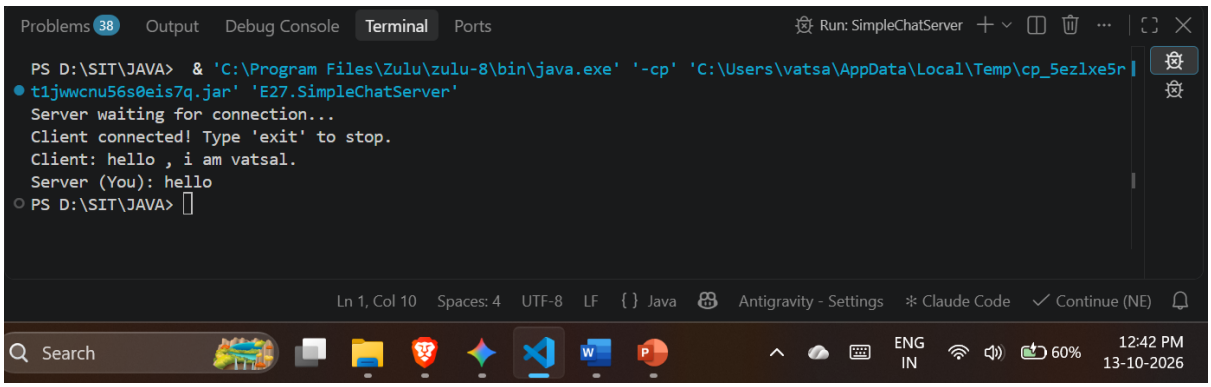
            String serverMsg = in.readLine();
            if (serverMsg == null || serverMsg.equalsIgnoreCase("exit")) {
                break;
            }
```

```
            System.out.println("Server: " + serverMsg);
        }

        s.close();
    }
}
```

Output:





2. Create a Java program using **socket programming** where the client sends a message to the server, and the server returns the **same message back (echo system)**.

Code:

SimpleEchoServer.java

```
package E27;

import java.lang.*;
import java.io.*;
import java.net.*;

public class SimpleEchoServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Echo Server is running...");
```

```
        Socket s = ss.accept();
        System.out.println("Client connected!");

                        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);

        String msg;
        while ((msg = in.readLine()) != null) {
            if (msg.equalsIgnoreCase("exit")) {
                break;
            }
            System.out.println("Received: " + msg);
            out.println("Server Echo: " + msg);
        }

        s.close();
        ss.close();
    }
}
```

SimpleEchoClient.java

```
package E27;

import java.lang.*;
import java.io.*;
import java.net.*;

public class SimpleEchoClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5000);
        System.out.println("Connected to server! Type 'exit' to stop.");

                        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
                        BufferedReader kb = new BufferedReader(new
InputStreamReader(System.in));

        String msg;
        while (true) {
            System.out.print("Enter message: ");
```

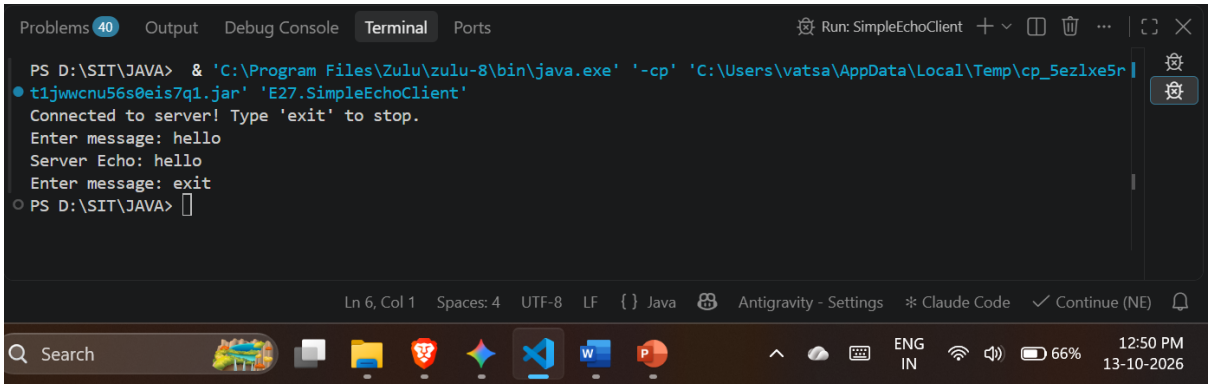
```
            msg = kb.readLine();
            out.println(msg);

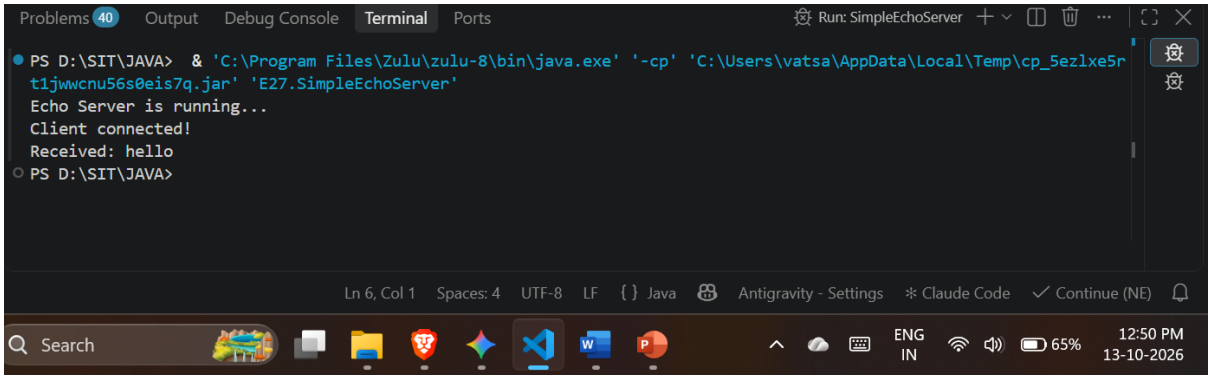
            if (msg.equalsIgnoreCase("exit")) {
               break;
            }

            System.out.println(in.readLine());
        }

        s.close();
    }
}
```

Output:





Conclusion:

In this experiment, I learned how to establish reliable two-way communication between applications using Java's Socket and ServerSocket classes over a TCP/IP network. I learned to effectively manage data transmission by utilizing BufferedReader and PrintWriter to read and write network streams seamlessly. I also learned how to implement continuous loops to create interactive chat and echo systems rather than just single-message exchanges. Finally, I learned the importance of proper resource management by configuring specific exit commands to safely terminate loops and close network connections.